

Министерство науки и высшего образования Российской Федерации

Пензенский государственный университет

Кафедра «Вычислительная техника»

ОТЧЁТ

по лабораторной работе №7

по курсу «Логика и основы алгоритмизации в инженерных задачах»

на тему «Обход графа в глубину»

Выполнили:
студенты группы 24ВВВ3
Майер В.С.
Савин А.В.

Приняли:
Юрова О.В.
Деев М.В.

Цель работы – ознакомиться с алгоритмом обхода графа в глубину и освоить его практическую реализацию.

Общие сведения.

Обход графа – одна из наиболее распространенных операций с графами. Задачей обхода является прохождение всех вершин в графе. Обходы применяются для поиска информации, хранящейся в узлах графа, нахождения связей между вершинами или группами вершин и т.д.

Одним из способов обхода графов является поиск в глубину. Идея такого обхода состоит в том, чтобы начав обход из какой-либо вершины всегда переходить по первой встречающейся в процессе обхода связи в следующую вершину, пока существует такая возможность. Как только в процессе обхода исчерпаются возможности прохода, необходимо вернуться на один шаг назад и найти следующий вариант продвижения. Таким образом, итерационно выполняя описанные операции, будут пройдены все доступные для прохождения вершины. Чтобы не заходить повторно в уже пройденные вершины, необходимо их пометить как пройденные.

Таким образом, можно предложить следующую рекурсивную реализацию алгоритма обхода в глубину.

Вход: G – матрица смежности графа.

Выход: номера вершин в порядке их прохождения на экране.

Алгоритм ПОГ

1.1. для всех i положим $NUM[i] = \text{False}$ пометим как "не посещенную";

1.2. **ПОКА** существует "новая" вершина v

1.3. ВЫПОЛНЯТЬ DFS (v).

Алгоритм DFS(v):

- 2.1. пометить v как "посещенную" $NUM[v] = \text{True}$;
- 2.2. вывести на экран v;
- 2.3. **ДЛЯ** i = 1 **ДО** size_G **ВЫПОЛНЯТЬ**
- 2.4. **ЕСЛИ** $G(v,i) == 1$ **И** $NUM[i] == \text{False}$
- 2.5. **ТО**
- 2.6. {
- 2.7. DFS(i);
- 2.8. }

Реализация состоит из подготовительной части, в которой все вершины помечаются как не помеченные (п.1.1) и осуществляется запуск процедуры обхода для вершин графа (п.1.2, 1.3). И непосредственно процедуры обхода, которая помечает текущую (т.е. ту, в которой на текущей итерации находится алгоритм) вершину как посещенную (п. 2.1). Затем выводит номер текущей вершины на экран (п.2.2) и в цикле просматривает v-ю строку матрицы смежности графа $G(v,i)$. Как только алгоритм встречает смежную с v не посещенную вершину (п.2.4), то для этой вершины вызывается процедура обхода (п.2.7).

Задание:

1. Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного графа G. Выведите матрицу на экран.

2. Для сгенерированного графа осуществите процедуру обхода в глубину, реализованную в соответствии с приведенным выше описанием.

Листинг

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <locale.h>
#include <conio.h>

void DFS(int** G, int numG, int* visited, int s) {
    visited[s] = 1;
    printf("%3d", s);
    for (int i = 0; i < numG; i++) {
        if (G[s][i] == 1 && visited[i] == 0)
            DFS(G, numG, visited, i);
    }
}

int isIsolated(int** G, int numG, int vertex) {
    for (int i = 0; i < numG; i++) {
        if (i != vertex && G[vertex][i] == 1) {
            return 0;
        }
    }
    return 1;
}

int main() {
    setlocale(LC_ALL, "Russian");
    int** G;
    int* visited;
    int numG;
    int current;

    srand(time(NULL));

    printf("Сколько вершин?\n");
    scanf_s("%d", &numG);

    G = (int**)malloc(numG * sizeof(int*));
    visited = (int*)malloc(numG * sizeof(int));
    for (int i = 0; i < numG; i++) {
        G[i] = (int*)malloc(numG * sizeof(int));
    }

    for (int i = 0; i < numG; i++) {
        visited[i] = 0;
        for (int j = i; j < numG; j++) {
            if (i == j) {
                G[i][j] = 0;
            }
            else {
                G[i][j] = G[j][i] = rand() % 2;
            }
        }
    }

    printf("\nМатрица смежности:\n");
    for (int i = 0; i < numG; i++) {
        for (int j = 0; j < numG; j++) {
            printf("%3d", G[i][j]);
        }
    }
}
```

```

        printf("\n");
    }

    printf("\nВведите номер стартовой вершины (0-%d): ", numG - 1);
    scanf_s("%d", &current);

    if (current < 0 || current >= numG) {
        printf("Ошибка: неверный номер вершины!\n");
    }
    else {
        if (isIsolated(G, numG, current)) {
            printf("\nВершина %d - изолированная\n", current);
        }
        else {
            printf("\nПуть обхода в глубину из вершины %d:", current);
            DFS(G, numG, visited, current);

            printf("\n\nПосещенные вершины: ");
            int visitedCount = 0;
            for (int i = 0; i < numG; i++) {
                if (visited[i]) {
                    printf("%d ", i);
                    visitedCount++;
                }
            }
            printf("\nВсего посещено вершин: %d из %d\n", visitedCount, numG);
        }
    }

    for (int i = 0; i < numG; i++) {
        free(G[i]);
    }

    free(G);
    free(visited);

    _getch();
    return 0;
}

```

Результат работы программы показан на рисунке 1

```

Сколько вершин?
10

Матрица смежности:
0 1 1 1 1 1 1 1 0 1
1 0 0 1 1 0 1 1 1 1
1 0 0 0 0 0 0 1 1 0
1 1 0 0 0 0 1 1 0 0
1 1 0 0 0 1 1 0 1 1
1 0 0 0 1 0 0 1 0 0
1 1 1 1 1 0 0 1 1 1
1 1 1 1 0 1 1 0 1 0
0 1 1 0 1 0 1 1 0 1
1 1 0 0 1 0 1 0 1 0

Введите номер стартовой вершины (0-9): 5

Путь обхода в глубину из вершины 5: 5 0 1 3 6 2 7 8 4 9

Посещенные вершины: 0 1 2 3 4 5 6 7 8 9
Всего посещено вершин: 10 из 10

```

Рисунок 1

Вывод: в ходе выполнения лабораторной работы ознакомились с алгоритмом обхода графа в глубину и освоили его практическую реализацию.